```
SEEKING_CHOICES = (
    ("M", "Musician"),
    ("B", "Band"),
)
```

Django also provides utility classes for building these kinds of structures. The `django.db.models.TextChoices` class allows you to achieve the same result by coding an enumeration-like class:

```
class SeekingChoice(models.TextChoices):
    MUSICIAN = "M"
    BAND = "B"
```

`TextChoices` classes use attribute names as labels with attribute values as the value in the database. All-capital attribute names get converted into sentence-case labels: the `MUSICIAN` attribute becomes the `"Musician"` label. The `.choices` attribute of this class contains the tuple-of-tuples structure required by the `choices` argument to `CharField`. There isn't much difference in the amount of code here, but the `TextChoices` class is more explicit about its purpose, and personally, I find it easier to read.

Nesting your choices

Some programmers prefer to make their choices structure a member of the model class that uses it. For example, `MusicianBandChoice` could be an inner class of `SeekingAd`. This is a matter of preference. Scoping it this way makes it clear where it gets used, but accessing it requires writing out the whole thing. Personally, I find `MusicianBandChoice.MUSICIAN` lengthy enough, and having to type `SeekingAd.MusicianBandChoice.MUSICIAN` feels like overkill.

When a band is seeking a musician, the `seeking` field is `"M"`, and you want to track which band is seeking a musician. You do this by populating the `band` foreign key. Likewise, when a musician is seeking a band, the `seeking` field is `"B"` and the `musician` foreign key should be populated with the musician seeking a band. This introduces a new wrinkle: the validation of the fields in your model are interdependent.

In chapter 4, you learned about basic ORM field validation, like disallowing empty values and constraints limiting what could be in a field. Most validation and constraints are field level, meaning inter-field dependencies don't come out of the box. The `Model` base class has a series of methods that are called to validate an object. One of them is called `.clean()`, and it can be overridden to verify inter-field relationships. The base class's `.clean()` method is empty, but you can add code to check for problems and raise a `ValidationError` if one is found. The following code is part of just such a validation process:

```
def clean(self):
    if self.seeking == MusicianBandChoice.MUSICIAN:
        if self.band is None:
```